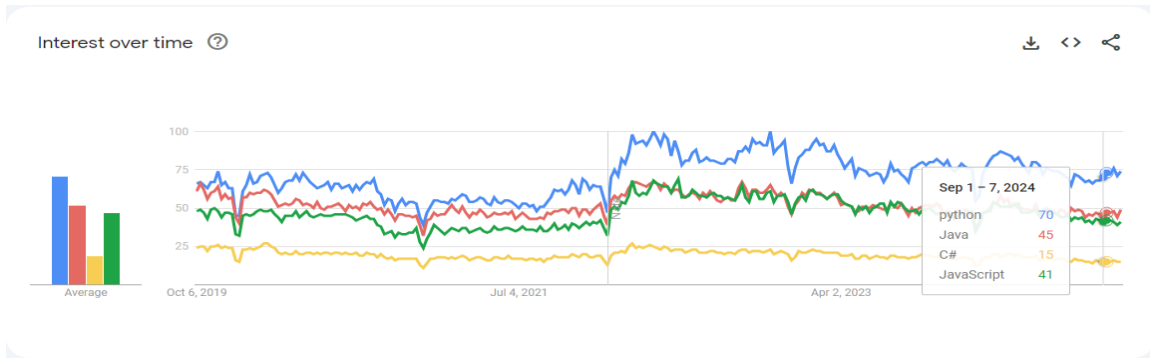


المحاضرة الأولى والثانية

1. مقدمة:

ماهي لغة البايثون؟



Ref: <https://trends.google.com/trends/>

مميزات لغة بايثون

1 | بايثون حرة و مفتوحة المصدر



- قراءة كود المصدر (Source code)
- القيام بالتعديل عليها
- إستخدامها في برمجيات جديدة
- توزيع نسخ من البرمجيات الجديدة

2 | بايثون بسيطة و سهلة التعلم

Java

```
public class HelloWorld
{
    public static void main(String[] args)
    {
        System.out.print("Hello, world!");
    }
}
```

Python

```
print("Hello, world!")
```

لاستخراج الأحرف الثلاث الأولى من كلمة: **YouTube**

JavaScript

```
str.substr(0,3)
```

C#

```
str.substring(0,3)
```

Python

```
str[0:3]
```



مجتمع بايثون (Python's community)
ضخم جدا. لذلك فالمصادر و الدروس
المجانية متوفرة بكل اللغات و لجميع
المستويات

3 | بايثون لغة قوية

Interpreted

Memory Management

Dynamically-Typed

Embeddable

Object-Oriented

Large Ecosystem

شركات كثيرة أصبحت تعتمد على بايثون في تنفيذ برامجها

3 | بايثون لغة قوية



Google



Facebook



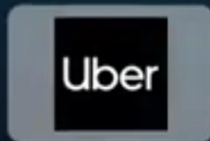
Amazon



Dropbox



Netflix



Uber

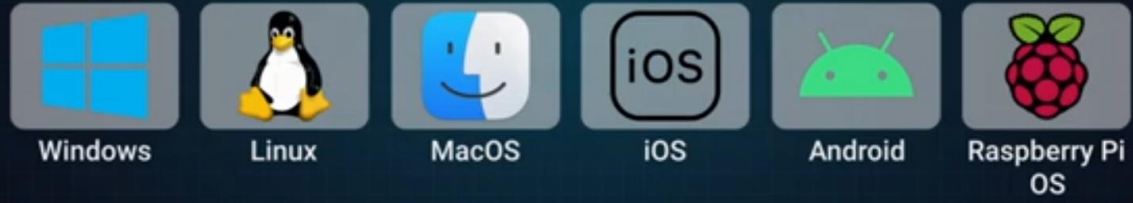


Reddit

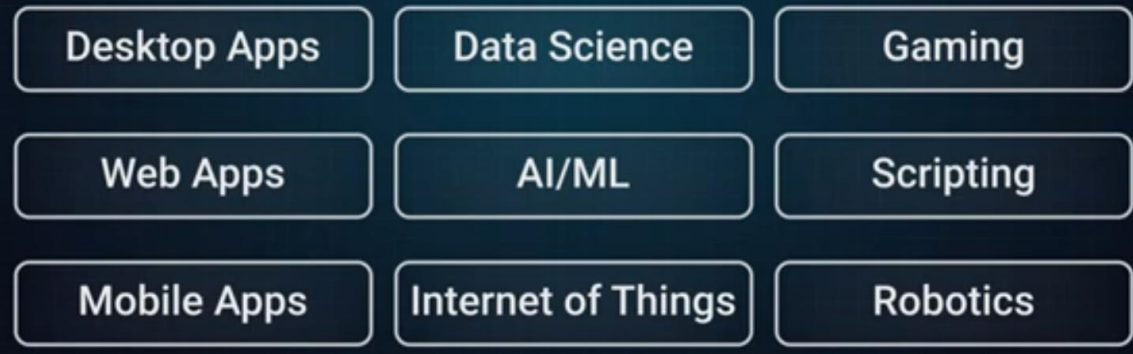


Spotify

4 | بايثون متعددة المنصات (Cross-platform)



5 | بايثون متعددة الإستخدامات



أجر مبرمج بايثون السنوي في الولايات المتحدة الأمريكية



مميزات لغة بايثون



1 | بايثون حرة و مفتوحة المصدر

2 | بايثون بسيطة و سهلة التعلم

3 | بايثون لغة قوية

4 | بايثون متعددة المنصات

5 | بايثون متعددة الإستخدامات

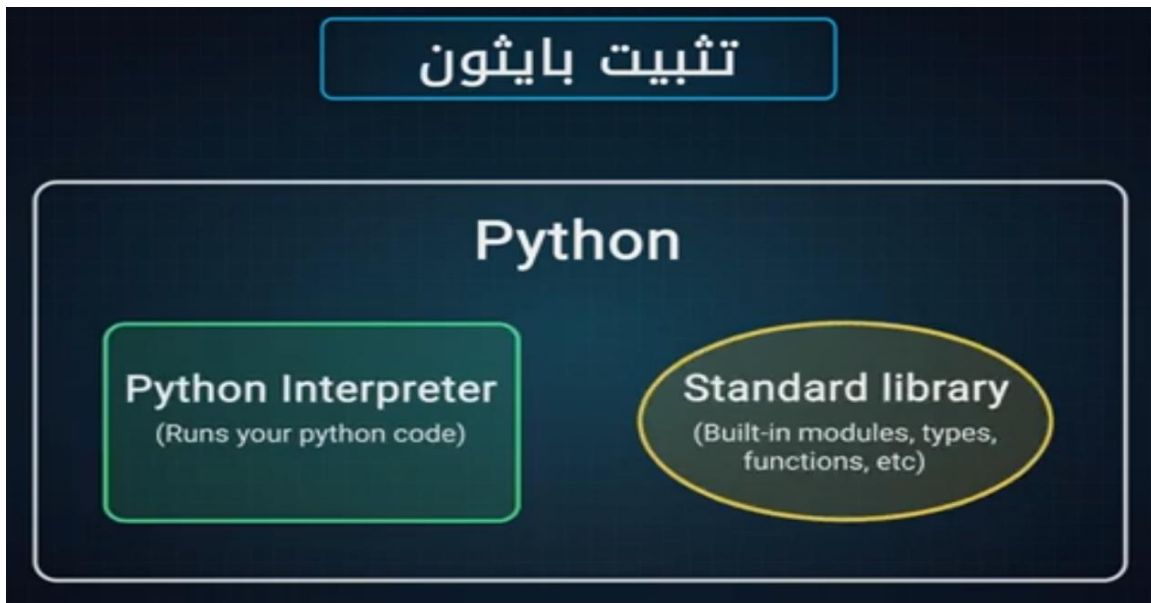
تاريخ لغة بايثون



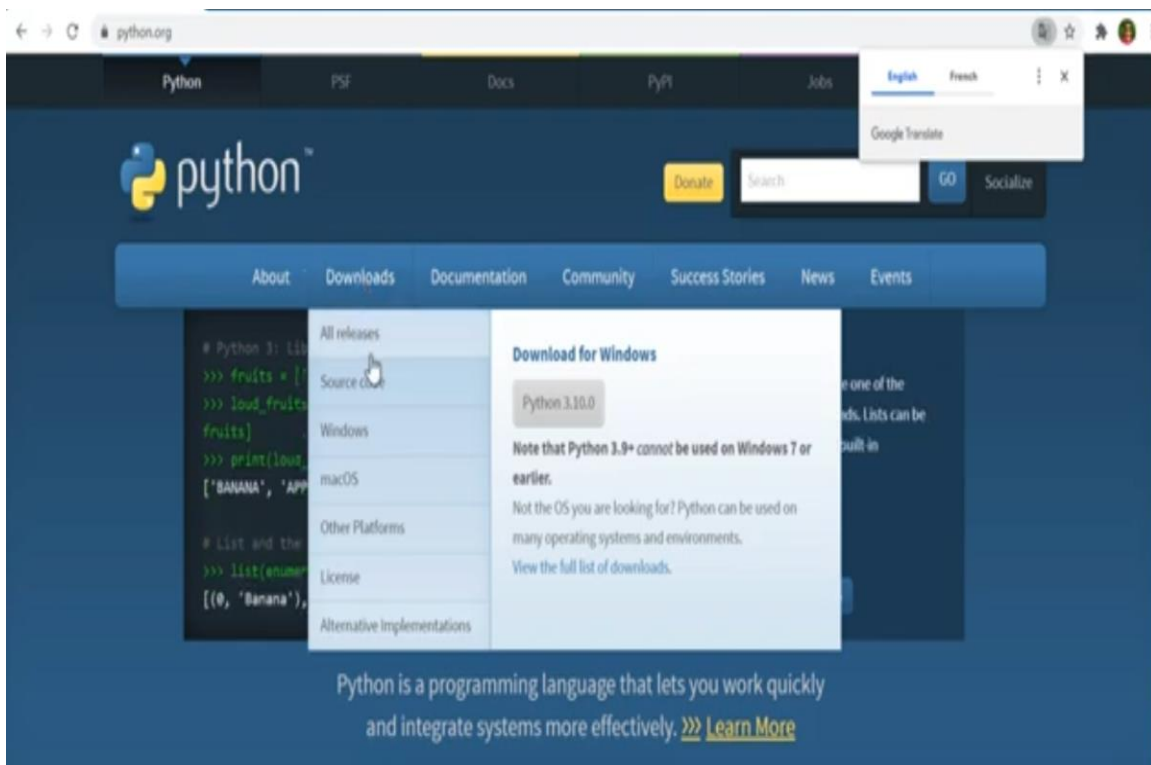
Install Python and Vs Code



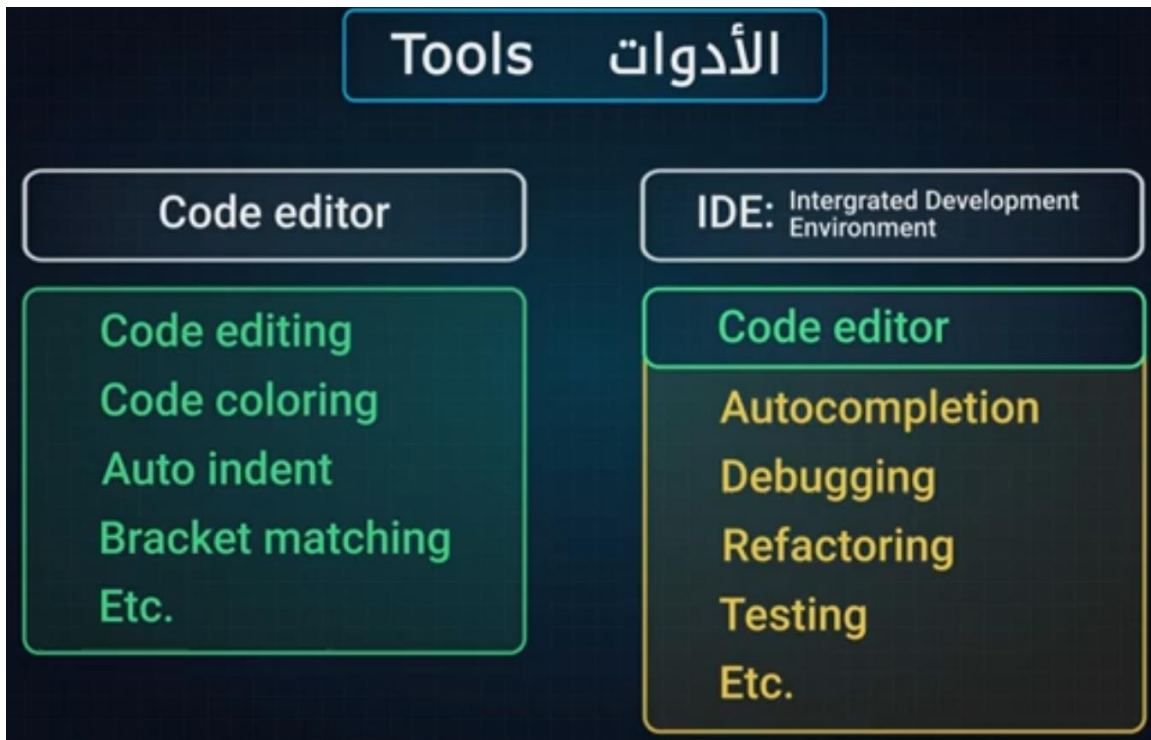
إصدار بايثون هو مفسر مع مكتبة أساسية



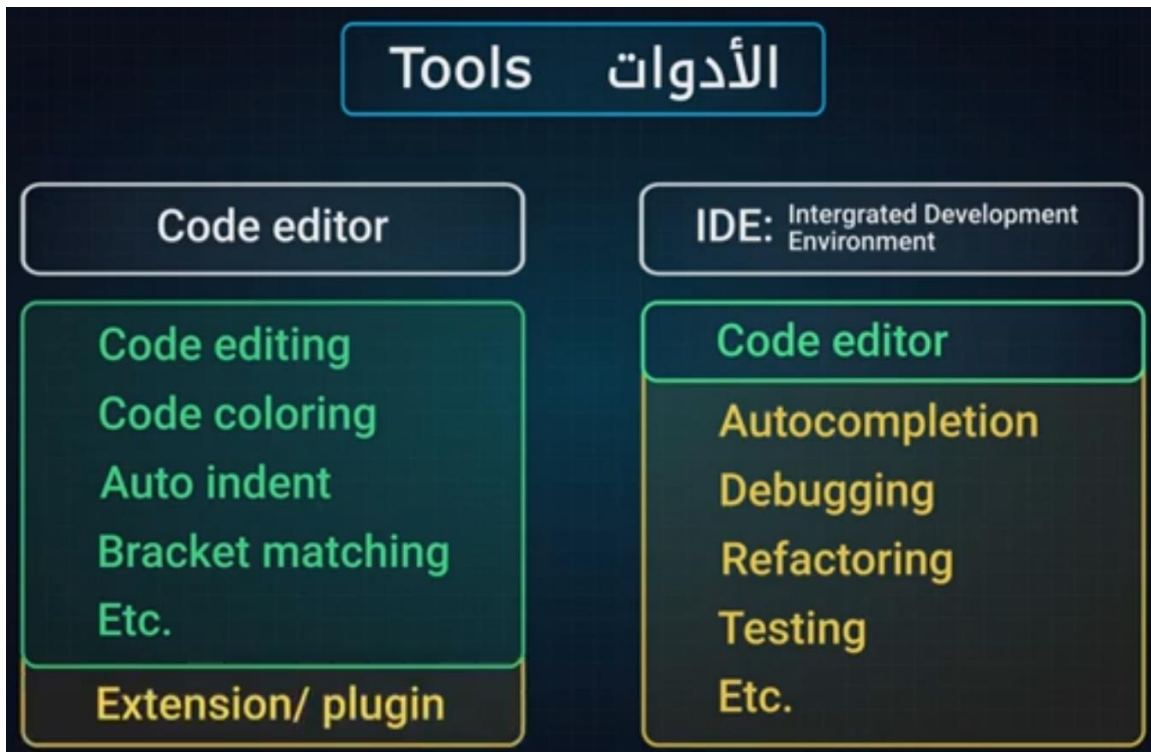
تنصيب بايثون لأجل ويندوز



الفرق بين محرر الكود وبيئة التطوير المتكاملة



يمكن تحويل محرر الكود إلى بيئة تطوير بإضافة بعض الامتدادات



أمثلة على محرر الكود وبيئة التطوير المتكاملة



يتم تثبيت VS Code من الموقع:

<https://code.visualstudio.com>

Or

<https://www.jetbrains.com/pycharm/download/?section=windows>

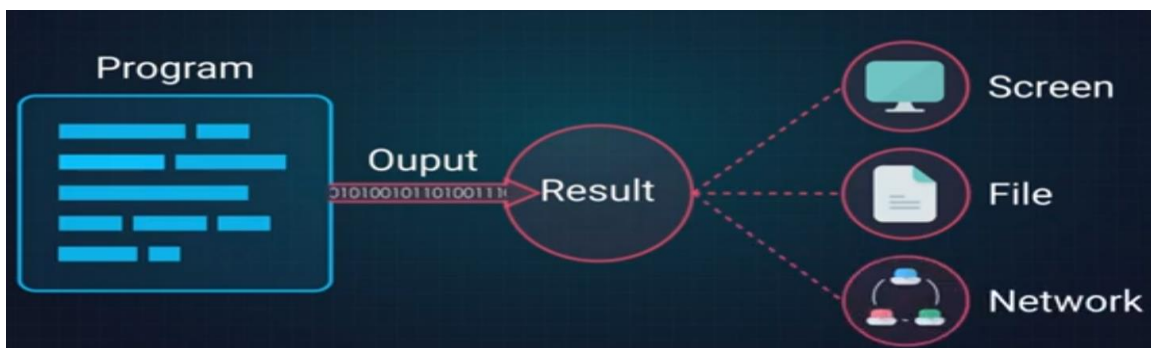
Or

<https://www.spyder-ide.org/download/?os=windows&arch=x64>

الإدخال والإخراج في بايثون (Input and output in Python):



طرق إخراج البيانات في بايثون



أسهل طريقة هي عرض البيانات على الشاشة باستخدام تعليمة (print)

نشاط 1

أكتب برنامج بلغة بايثون يعرض الرسالة التالية على الشاشة.

Hello, everybody!

Programming is a useful skill.

It's the right time to learn python programming.

Bye for now :)

```
message.py
1 print('Hello Everybody!')
2 print('Programming is useful skill.')
3 print("It's the right time to learn python programming")
4 print("Bye for now :)")
```

Syntax

```
print(value, ...)
```

e Example

```
print('Hello world, single quotes')
```

```
print("Hello world, double quotes")
```

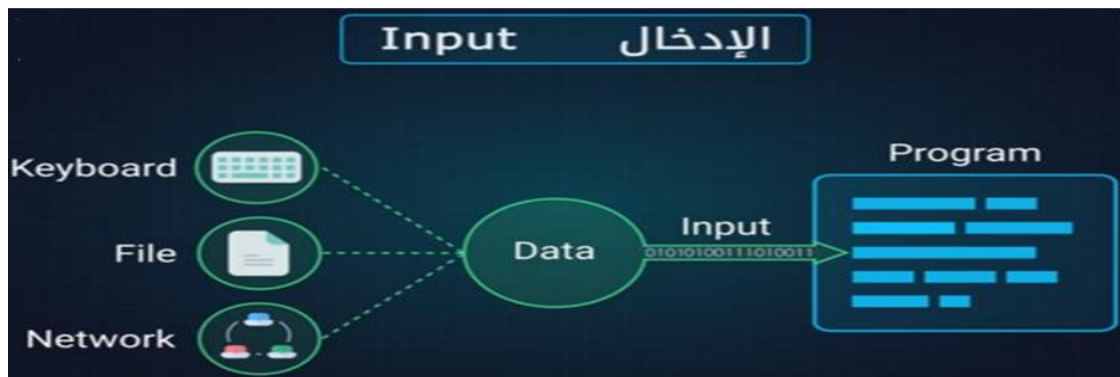
تمرين 1

أكتب برنامج بلغة بايثون مهمته طباعة بطاقة المستخدم (User Card) التالية على الشاشة.

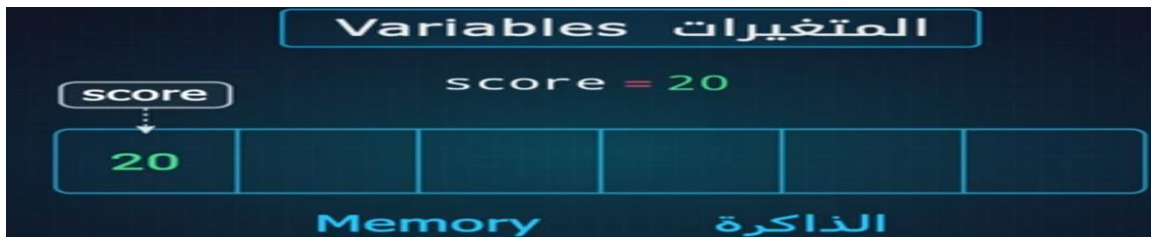
```
***** User Card *****
* Name: Guido van Rossum   *
* Title: Computer programmer *
* Date of birth: 31/01/1956 *
*****
```

```
user card.py
1 print("***** User Crad *****")
2 print("* Name: Guido van Rossum   *")
3 print("* Title: Computer programmer *")
4 print("* Date of birth: 31/10/1956 *")
5 print("*****")
```

طرق إدخال البيانات في بايثون



المتغيرات في بايثون (Variables in Python)



نشاط 1

- قم بتعريف المتغيرات التالية:
 - متغير اسمه player_name و أعطه القيمة "Ahmed"
 - متغير اسمه age و أعطه القيمة 20
 - متغير اسمه score و أعطه القيمة 30.5
 - متغير اسمه win و أعطه القيمة False
- قم بتغيير قيمة المتغير win إلى True
- أضف 10 إلى قيمة المتغير score
- أطبع قيم المتغيرات بشكل مرتب على الشاشة

```
variables.py > ...
1  player_name = "Ahmed"
2  age = 20
3  score = 30.5
4  win = False
5  win = True
6  score = score + 10
7  print("Player name:", player_name)
8  print("Age:", age)
9  print("Score:", score)
10 print("Win:", win)
```


Syntax

Assignment operator
علامة إسناد

variable_name = value

Example

```

x = 15
y = 20
total = x + y
print(type(total)) <class 'int'>

```

قواعد تسمية المتغيرات

عند تسمية المتغيرات يجب مراعات هذه القواعد :

- يجب ان يبدأ إسم المتغير بحرف أو بعلامة underscore (_)
- يجب أن يحتوى إسم المتغير فقط على حروف و أرقام إضافة إلى علامة underscore (_)
- يجب ألا يحتوي إسم المتغير على مسافات spaces و يمكن إستبدال space بإستخدام underscore مثل student_number
- يجب ان لا يكون المتغير بإسم كلمة من الكلمات المحجوزة مثل import ,print ,True وغيرها

تمرين 1

ضع علامة صح ✓ أمام أسماء المتغيرات الصحيحة و علامة خطأ ✗ أمام الأسماء الخاطئة

✗ True	✓ _x
✓ Grade75	✗ price\$
✗ Birth date	✗ 3color
✓ UserCategory	✓ student_name



Note

snake_case_convention

1 | Variables

```
age = 20
```

```
student_name = "Ahmed"
```

2 | Constants

```
PI = 3.14
```

```
GRAVITY = 9.8
```

تمرين 2



(1) قم بتعريف المتغيرات التالية:

- متغير اسمه `full_name` و أعطه القيمة "Bill Gates"

- متغير اسمه `country` و أعطه القيمة "United States"

(2) إستخدم المتغيرات السابقة لتعرض النتيجة التالية على الشاشة:

```
Hello Bill Gates
```

```
You are from United States
```

(3) قم بتغيير البرنامج حتى يعرض معطياتك الشخصية

```
personal info.py > ...  
1 full_name = "Bill Gates"  
2 country = "United States"  
3 print("Hello", full_name)  
4 print("You are from", country)
```

نشاط 1

غير البرنامج personal_info.py لتصبح مهمته:

(1) إدخال إسم المستخدم من لوحة المفاتيح و إسناده للمتغير

full_name

(2) إدخال بلد المستخدم من لوحة المفاتيح و إسناده للمتغير country

(3) طباعة النتيجة على الشاشة

مثال: لو أدخل المستخدم الإسم Mohamed Arabi و البلد Egypt

فإن النتيجة التالية ستظهر على الشاشة:

```
Hello Mohamed Arabi  
You are from Egypt
```

```
personal_info.py > ...  
1 full_name = input("Enter your name: ")  
2 country = input("Enter your country: ")  
3 print("Hello", full_name)  
4 print("You are from", country)
```

Syntax

The message is printed on
the terminal before the input

```
variable_name = input(message)
```

Example

```
email_address = input("What's your email address?")  
print(email_address)
```

تمارين

ضع علامة صح ✓ أمام الإجابة الصحيحة و علامة خطأ أمام الإجابة الخاطئة ✗

الأمر الذي تحتاجه لعرض المجموع (total):

✗ `input("Type the total")`

✗ `print(a,b)`

✓ `print("The sum is", total)`

ضع علامة صح ✓ أمام الإجابة الصحيحة و علامة خطأ أمام الإجابة الخاطئة ✗

إسناد القيمة 10 للمتغير x:

✗ `x ← 10`

✓ `x = 10`

✗ `10 = x`

ضع علامة صح ✓ أمام الإجابة الصحيحة و علامة خطأ أمام الإجابة الخاطئة ✗

الأمر الذي تحتاجه لكي تطلب من المستخدم إدخال قيمة المتغير name:

✗ `name = "Ahmed"`

✓ `name = input("Enter the name:")`

✗ `print("Type the name:")`

ضع علامة صح ✓ أمام الإجابة الصحيحة و علامة خطأ أمام الإجابة الخاطئة ✗

الإسم الذي يعتبر إسما لمتغير (variable):

✗ `2box`

✗ `a/2`

✗ `product type`

✓ `Customer_ID`

ضع علامة صح ✓ أمام الإجابة الصحيحة و علامة خطأ ✗ أمام الإجابة الخاطئة ✗

نتيجة البرنامج بالحسابات اليدوية؟

✗ 13 25

✗ 25 13

✓ 25 25

✗ 13 13

```
a = 13
b = 25
a = b
print(a, b)
```

التعليقات

Comments التعليقات

1 | التعليقات السطرية

```
# This is a comment
```

2 | التعليقات متعددة الأسطر

```
""" This is a comment
written in more than
just one line """
```

نهاية المحاضرة الأولى

المحاضرة الثانية

العنوان

1. التعليمات الأساسية ومقابلاتها في بايثون
 2. الأنماط الأساسية
 3. فقط في بايثون
 4. التمارين
- الكلمات المحجوزة المستعملة
- الكلمات المحجوزة في بايثون
- توصيف عمل الكلمات المحجوزة في بايثون
- التوابع مسبقا التعريف المستعملة
- التوابع المسبقة التعريف في بايثون

المفاهيم الأساسية

- التعليمات الأساسية: if, if-else, while, for
- الأنماط الأساسية: int, float, bool, string

أهداف تعليمية

- يتعرف الطالب في هذا الفصل على:
- بنية النص البرمجي بلغة بايثون.
 - العمليات الهامة التي لم يتاح المجال لإعطائها في البرمجة 1-2 مثل العمليات على البتات bitwise operations.

1. التعليمات الأساسية ومقابلاتها في بايثون

سنتعرف عبر المثال على التعليمات الأساسية في بايثون.

التعليمات الأساسية هي 5 تعليمات:

القراءة، الكتابة، إسناد قيمة تعبير إلى متحول، التعليمة الشرطية، التعليمة التكرارية.

هذه التعليمات كافية مبدئياً لكتابة البرامج التي نريد.

- **تنويه:** يعتبر أي استدعاء "لوظائف جاهزة" (البرامج الجزئية المتاحة في اللغة) هو تعليمة، ويمكن اعتبار تعليمتي القراءة والكتابة هي استدعاء لبرامج جزئية تقوم بذلك.

برنامج حساب القاسم المشترك الأعظم وفق خوارزمية إقليدس.

(تكرار طرح الأصغر من الأكبر ما دام العددين غير متساويين).

Python

```
#Read a, b
Sa = input("a="); a = int(Sa)
Sb = input("b="); b = int(Sb)

#Compute GCD of a and b
while a != b:
    if a > b:
        a = a - b
    else:
        b = b - a
g = a
print("GCD = " + str(g))
```

جدول التعليمات:

<pre>Sa = input("input a :"); a = int (Sa) ; Sb = input("input b :"); b = int (Sb) ; a = int (input("input a :")) ;</pre> لا يوجد تعريف لنمط المتحول في Python، وإنما تقوم عملية الإسناد بتعريف نمط المتحول ضمناً.	1. تعليمة القراءة: أي إدخال معطيات وتكتب على أحد الأشكال
<pre>print("GCD =" , g); print("GCD =" + str(g));</pre>	2. تعليمة الكتابة: أي إظهار النتائج
<pre>a = a - b</pre>	3. تعليمة الإسناد: أي وضع قيم تعبير في متحول
<pre>if a > b: a = a - b else a = b - a</pre> تحل الإزاحة محل الأقواس، فكل الأسطر ذات نفس درجة الإزاحة موجودة ضمن كتلة تعليمات واحدة.	4. التعليمة الشرطية
<pre>while (a != b): DoSomething()</pre>	5. التعليمة التكرارية

- **تنويه:** لاحظ: بعد كل تعليمة

if, if-else, while ...

- **تنويه:** تُحدّد كتلة التعليمات الخاصة بالتعليمة بالمحاذاة **alignment**

التعليقات :Comments

#

تبدأ التعليقات في Python بمحرف التهشير (#)، وتمتد حتى نهاية السطر الحالي. يمكن أن تظهر التعليقات في بداية سطر أو بعد فراغ أو تعليمة، ولكن ليس ضمن سلسلة محرفية. أي أن محرف التهشير (#) داخل سلسلة محرفية هو فقط محرف تهشير ولا يؤدي إلى تعليق. بما أننا نستخدم التعليقات لتوضيح النص البرمجي فهي لا تُنفَّذ، ويمكن حذفها أثناء إعادة كتابة المثال من أجل تنفيذه دون أن يؤثر الحذف على التنفيذ.

```
# This is a comment
```

""" تعليق مجموعة أسطر """

```
"""
Python is an interpreted, high-level and general-purpose
programming language. Created by Guido van Rossum and first
released in 1991
"""
```

تعليلة القراءة أو إدخال المعطيات.

1.1. تعليلة القراءة

تقوم التعليلة **input()** بقراءة دخل على شكل سلسلة نصية. للتحويل إلى النمط الأساسي الذي نريد، نستخدم اسم النمط الأساسي (**int**, **float**, **bool**).

- عدد صحيح باستخدام التعليلة **int()**.
- عدد حقيقي باستخدام التعليلة **float()**.
- قيمة منطقية (**True**, **False**) باستخدام التعليلة **bool()**.

```
# read integer
Sa = input("input integer"); a = int(Sa)
# read float
Sb = input("input float"); b = float(Sb)
# read Boolean
Sc = input("input boolean"); c = bool(Sc)
```

2.1. تعليمة الكتابة

تقوم تعليمة `print()` بكتابة قيم المعاملات المعطاة إليه. يقبل تابع `print()` عدّة معاملات، ويمكن أن تكون المعاملات من أنماط مختلفة مثل أعداد بفاصلة عشرية وسلاسل محرفيّة `strings`.

- تفصل بين المعاملات المختلفة بالفاصلة (,).

في حال كان المعامل سلسلة نصية فيمكن أن ندمج معه معاملات أخرى من أنماط مختلفة لإظهارها، بشرط تحويلها إلى سلسلة نصية (إذا لم تكن كذلك). يتم التحويل إلى سلسلة نصية باستخدام التعليمة `.str()`.

مثال:

```
a=5; b=7; c=8; g=1;
print("a , b, c" , a, b, c);
print("GCD =" + str(g));
```

3.1. تعليمة الإسناد

مثال:

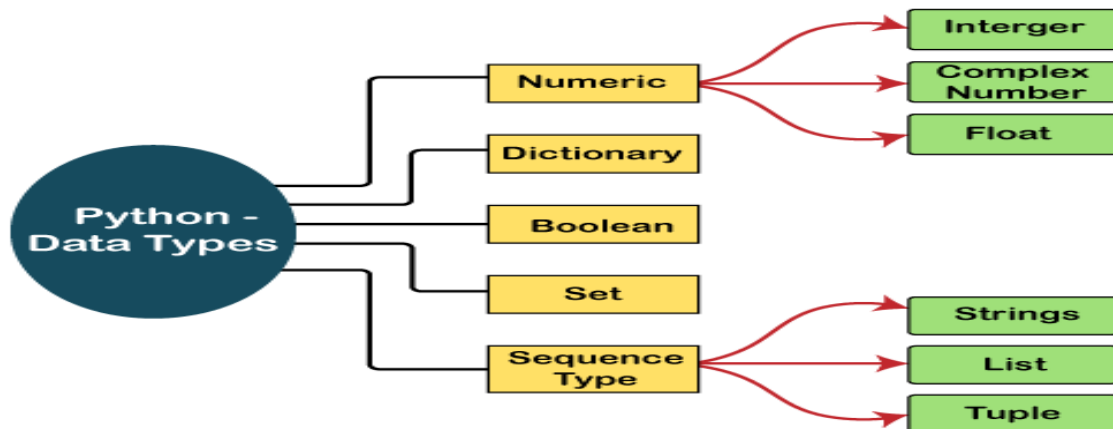
Python

```
a = b = c = d =8    # Multiple Assignment
```

تأخذ `a, b, c, d` قيم جديدة هي 8 ولكن ضمن عملية إسناد متعدّدة.

- تقوم عملية الإسناد بتحديد نمط المتحول المستخدم بشكل ضمني (`implicitly`).
- يمكن أن يتغير نمط المتحول عند كل عملية إسناد جديدة لهذا المتحول.

الأنماط الأساسية للمتغيرات في بايثون (Type of Variables)



أنماط بنى المعطيات المضمنة في بايثون في الحالة العامة (Default Python Built-in Data-Types):

Ext. Type:	str
Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	dict
Set Types:	set, frozenset
Boolean Type:	bool
Binary Types:	bytes, bytearray, memoryview
None Type:	None Type

الإسناد المزود بعملية (Augmented Assignment Operation):

لتكن **op** إحدى العمليات التالية: {+, -, *, /, &, |}.
 من الممكن أن نستخدم إسناد مزود بعملية من العمليات السابقة له الشكل:

$$x \text{ op} = y$$

 بشكل مكافئ للإسناد البسيط التالي:

$$x = x \text{ op } y$$

Python

```
a, b = 56
a = -8
a += b
b *= 3
```

العمليات الحسابية والتعابير الرياضية:

تستخدم العمليات الحسابية ((), **, *, /, %, +, -) لتوليد التعابير الرياضية Expressions.

نلاحظ أثناء استخدام هذه العمليات ما يلي:

1. نتيجة جمع أو طرح أو ضرب عددين صحيحين هو عدد صحيح. أما إذا كان أحد العددين حقيقياً float فتكون النتيجة هي عدد حقيقي.
2. نتيجة استخدام عملية القسمة / لعدد على عدد آخر هي عدد حقيقي، ويمكن الحصول على ناتج القسمة "الصحيحة" لعدد صحيح على عدد صحيح باستبدال عملية القسمة (/) بالعملية (//).
3. تنفذ العمليات الحسابية في التعابير الرياضية الكبيرة وفق الأوليات التالية:

- ()
- ** رفع لقوة
- *
- /
- %
- +
- -

Conditional statements العبارات الشرطية

if statement

if  then 

if `grade ≥ 60` then `Pass!`

Syntax

```
if condition :  
    statement 1  
    statement 2  
    ...  
following_statement
```

Boolean expression evaluates to True or False

This block is executed if the condition is True

The block must be indented (4 spaces)

تمرين 1

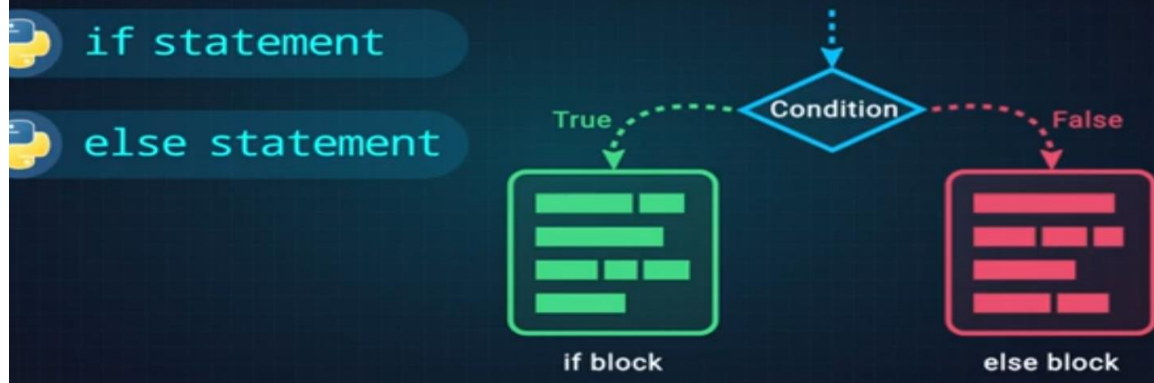
أكتب برنامج بلغة بايثون يطلب من المستخدم علامة الطالب من 100، ثم يطبع الرسالة:

إذا كانت علامة الطالب
تفوق 60

Congratulations!
Student is declared pass

```
student result.py > ...  
1  mark = float(input("Enter the student mark: "))  
2  if mark >= 60 :  
3      print("Congratulations!")  
4      print("Student is declared pass")  
5  else:  
6      print("Sorry")  
7      print("Student is declared fail")  
8
```

العبارات الشرطية Conditional statements



4.1. التعليمة الشرطية if

للتعليمة الشرطية في بايثون ثلاث أشكال:

الشكل if

```
if expression:  
    instructionBlock
```

- يجري اختبار الشرط المنطقي **expression** الذي يعيد قيمة منطقية صح أو خطأ.
- فإذا كانت صح يجري تنفيذ **instructionBlock**.
- وإلا يجري الخروج دون تنفيذ **instructionBlock**.

الشكل if else

```
if (expression):  
    instructionBlock1  
else:  
    instructionBlock2
```

يجري اختبار الشرط المنطقي **expression** الذي يعيد قيمة منطقية صح أو خطأ، فإذا كانت صح يجري تنفيذ **instructionBlock1**، وإلا يجري تنفيذ **instructionBlock2**.

Syntax

```
if condition :  
    statement 1  
    statement 2  
    ...  
else :  
    statement 1  
    statement 2  
    ...
```

This block is executed if the condition is True

This block is executed if the condition is False

الشكل if elif

```
if (expression1):  
    instructionBlock1  
elif(expression2):  
    instructionBlock2
```

يجري اختبار الشرط المنطقي **expression1** الذي يعيد قيمة منطقية صح أو خطأ، فإذا كانت صح يجري تنفيذ **instructionBlock1**، وإلا يجري اختبار الشرط المنطقي **expression2**، فإذا كانت قيمته صح يجري تنفيذ **instructionBlock2**.

تعليلة الإسناد الشرطية:

يمكن في بعض الحالات عندما يكون الهدف من التعليلة الشرطية تنفيذ عملية إسناد أن نبني تعليلة شرطية مختصرة لها الشكل القواعدي التالي في C-Like:



في حال تحقق الشرط المنطقي المحدد في **expression**

يجري إرجاع **Value1**

وإلا يجري إرجاع **Value2**

وتكافئ التعليلة السابقة التعليلة:

Python

```
A = Value1 if expression else Value2
```

العامل الشرطي الثلاثي Ternary operator



if-else statement

4 lignes



Ternary operator

1 ligne

مثال: ماذا تمثل تعليلة الإسناد الشرطية التالية:

Python

```
x=-15  
absx = x if (x>=0) else -x  
print(absx)
```

الجواب (تابع القيمة المطلقة لـ x).

Syntax

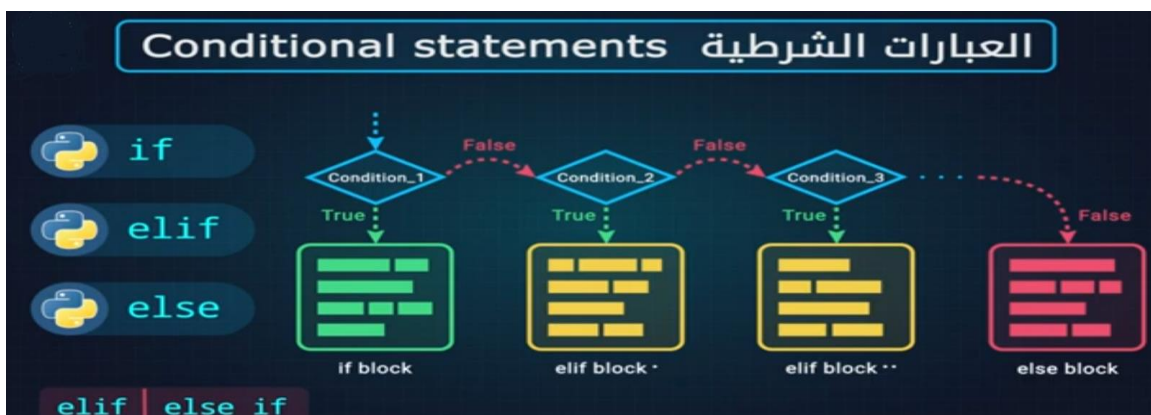
```
var_name = exp_1 if condition else exp_2
```

Example

```
a = 10
b = 20
max = a if a > b else b
print(max) # Get 20
```

نشاط: المطلوب كتابة برنامج بلغة بايثون بأكثر من طريقة وملاحظة الفرق من حيث عدد الأسطر البرمجية:

أكتب برنامج بلغة بايثون يطلب من المستخدم إدخال عدد صحيح، ثم يقرر إذا كان العدد المدخل هو عدد زوجي (Even) أو عدد فردي (Odd)
 مثال: عند إدخال العدد 6 يعرض البرنامج النتيجة "Even"
 عند إدخال العدد 15 يعرض البرنامج النتيجة "Odd"



نشاط: المطلوب مناقشة الكود البرمجي التالي:

```
elif statement.py > ...
1 temp = float(input("Enter a temperature:"))
2 if temp < 20 :
3     print("It's cold")
4 elif temp <= 33 :
5     print("It's nice")
6     print("Go for a walk outside")
7 else:
8     print("It's warm")
9     print("Drink water")
10 print("Done")
```




Syntax

```
if condition_1 :  
    Block_1  
elif condition_2 :  
    Block_2  
elif condition_3 :  
    Block_3  
...  
else :  
    Block_n  
following_statement
```

نشاط:

أكتب برنامج بلغة بايثون لإنشاء آلة حاسبة بسيطة:

- تطلب من المستخدم إدخال عددين و أحد العوامل الحسابية التالية

/ * - +

- تعيد لنا الناتج، و في حالة إدخال أي رمز لعامل آخر تخبرنا بأنه غير موجود

مثال:

Result	n2	op	n1
5.5	2.5	+	3
2	4	/	8
Unknown operator	7	%	5

الحلقات التكرارية Loops



while loop



for loop

Block of code

Block of code

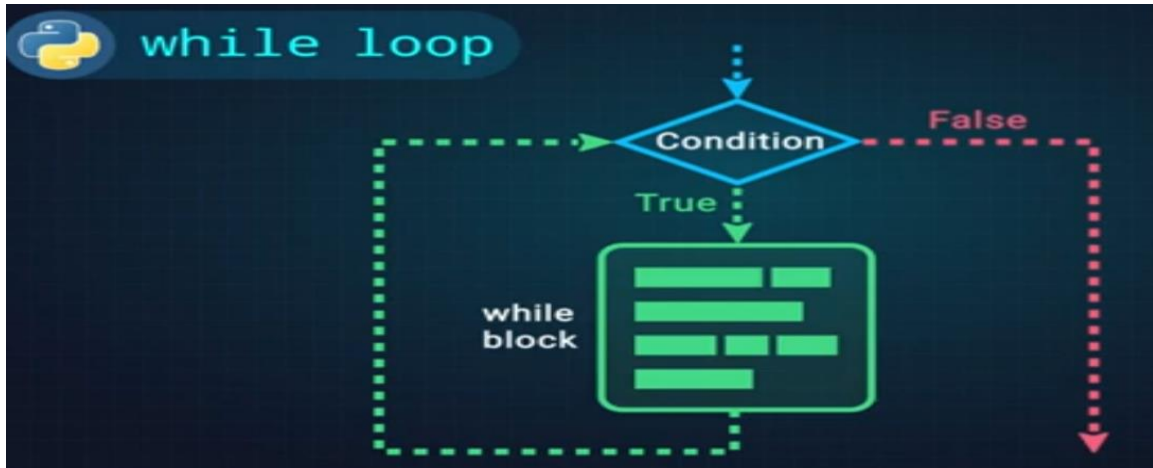
Block of code

Block of code

=

Loop

Block of code



5.1. التعليمات التكرارية while

يكون لتعليمات التكرار الحلقية while الشكل القواعدي التالي:



نشاط 1

أكتب برنامج بلغة بايثون يعرض الأرقام الفردية الأقل من 10

</>
Syntax

```

while condition:
    statement 1
    statement 2
    ...
else:
    statement 1
    ...
following_statement

```

statement 1

statement 2

...

This block is executed as long as condition is True

statement 1

...

This block is executed if the condition becomes False

following_statement

نشاط:

أكتب برنامج بلغة بايثون:

- يطلب من المستخدم إدخال مجموعة من الأعداد الموجبة، و يتوقف عند إدخال أول عدد سالب
- في نفس الوقت يقوم بحساب المجموع و عرضه على الشاشة

نشاط: مناقشة الكود البرمجي التالي بلغة بايثون

```
sum of numbers.py > ...
1  sum = 0
2  x = int(input("Enter a number: "))
3  while x >= 0 :
4      sum += x
5      print(sum)
6  x = int(input("Enter a number: "))
7  print(f"The sum is {sum}")
```

نشاط: مناقشة الكود التالي:

```
infinite loop.py > ...
1  is_learning = True
2  while is_learning :
3      print("You're learning!")
```

ملاحظة: لإنهاء الحلقة اللانهائية نضع مؤشر الماوس في الترمينال ونضغط (Ctrl+c).

6.1. التعليمة التكرارية بعدد for

مثال 1: حساب مجموع الأعداد 1, 2, 3, ..., M حيث M تعطى كدخل.

Python

```
#Read M
print(" Input M : ");
SM = input(); M = int(SM);
#Variables initialization
s = 0
#compute the Sum in s
for i in range(1,M+1):
    s = s + i;
#Write s
print("1+2+...+" , M , " = " , s);
```

- لاحظ كي نحدد قيمة نهاية عداد التكرار بالقيمة M يجب تحديد مجال التكرار `range(1,M+1)`.

الحلقات التكرارية Loops



for loop

string

P y t h o n

list

Sami Nour Ali Zied Farah

range

0 1 2 3 4 5 6



Note

Iterable

range
str
list
tuple
array

set
dict



Syntax

It takes items from iterable
one by one

```
for var in iterable:
```

```
    statement 1  
    statement 2  
    ...
```

This block is executed one for
each item in iterable

```
else:
```

```
    statement 1  
    ...
```

This block is executed
if the loop terminates naturally

```
following_statement
```


أكتب برنامج بلغة بايثون:

- يطلب من المستخدم إدخال رقم
- يطبع جدول الضرب الخاص بهذا الرقم. مع العلم ان جدول الضرب من 1 إلى 10

مثال: الرقم المدخل 6 | النتيجة:

6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
6 x 10 = 60

```

Multiplication table.py > ...
1  number = int(input("Enter an integer: "))
2
3  for count in range(1, 11) :
4      product = number * count
5      print(f"{number} x {count} = {product}")

```

حالات استخدام Use cases

 **for loop**

It is used when the number of iterations is known

1 2 3 ... n

 **while loop**

It is used when the number of iterations is not known

```

graph TD
    Start(( )) --> Condition{Condition}
    Condition -- True --> Block[Block of code]
    Block --> Condition
    Condition -- False --> End(( ))
    
```

أكتب برنامج بلغة بايثون:

- يطلب من المستخدم إدخال مجموعة من الأسماء و يخزنها في قائمة إسمها names (لا يتوقف إلا عند إدخال الكلمة quit)
- يقوم بترتيب القائمة names ترتيبا تصاعديا
- يعرض القائمة names طوليا (كل إسم في سطر)

مناقشة كود النشاط السابق:

```

list of names.py > ...
1  names = []
2  name = input("Enter a name: ")
3  while name.lower() != "quit" :
4      names.append(name)
5      # print(names)
6      name = input("Enter a name: ")
7
8  names.sort()
9
10 for name in names :
11     print(name)

```

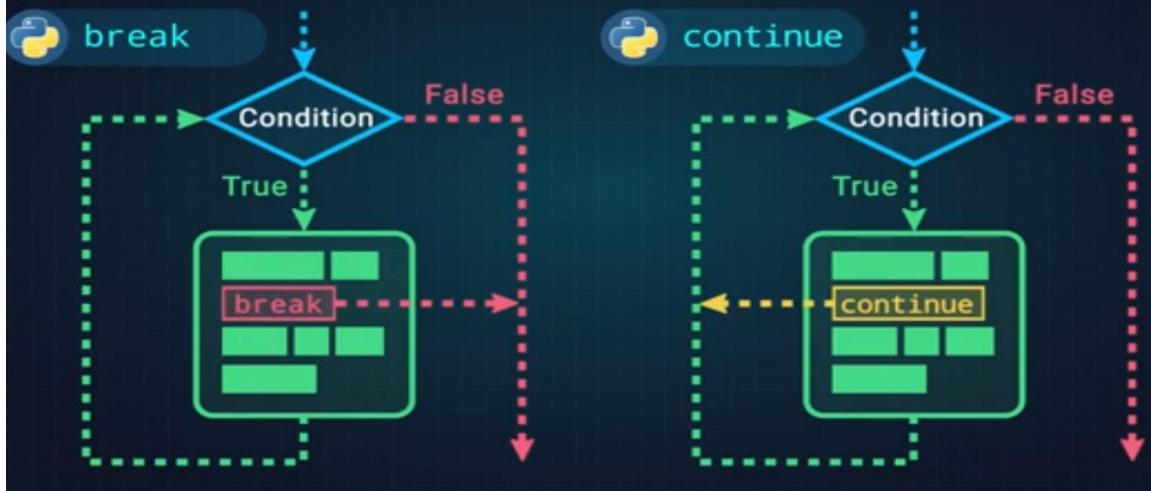
7.1. كسر البرمجة المهيكلية في بايثون

تؤدي التعليمة **break** إلى الخروج إلى ما بعد نهاية كتلة تعليمات.

مثال:

مثال بسيط يهدف فقط إلى توضيح استخدام التعليمة **break** للخروج من كتلة تعليمات. المطلوب من البرنامج تكرار تنفيذ تعليمات ما دامت قيم i أصغر أو يساوي قيمة معينة (القيمة 13 في مثالنا). وللتبسيط أكثر في مثالنا، تقتصر التعليمات المطلوب تكرار تنفيذها على تعليمة إظهار قيمة i . نريد إيقاف التنفيذ/ التكرار التعليمات لقيم i من مضاعفات 4. وما نقصده هنا بمضاعفات 4، هو أي عدد أكبر من 4، ويقسم 4 بلا باقي (أي باقي قسمته على 4 تساوي 0).

جمل التحكم Control statements



Syntax

<pre> for var in iterable: statement 1 statement 2 ... if condition: break ... statement n following_statement </pre>	<pre> for var in iterable: statement 1 statement 2 ... if condition: continue ... statement n following_statement </pre>
---	--

<pre> while condition: statement 1 statement 2 ... if condition: break ... statement n following_statement </pre>	<pre> while condition: statement 1 statement 2 ... if condition: continue ... statement n following_statement </pre>
---	--

نشاط: ما خرج الكود التالي:

```
control statements.py > ...
1 watches = ["ok", "ok", "ok", "faulty", "ok", "ok"]
2
3 # break
4 for status in watches:
5     if status == "faulty":
6         print("Stopping the production line!")
7         break
8     print(f"This watch is {status}")
```

نشاط: ما خرج الكود التالي:

Python

```
i = 0; n = 13
while (i <= n):
    if ((i % 4 == 0) and (i > 4)):
        break
    print(" i = " , i)
    i = i + 1
```

التعليمة continue في بايثون

تُستخدم تعليمة **continue** للاستمرار في التنفيذ ضمن الحلقة التكرارية، ولكن بالانتقال إلى التكرار التالي. نريد القيام بنفس العمل الذي قمنا به عند استخدام **break** في المثال السابق. أي تجنب إظهار مضاعفات 4، ولكن أريد لبرنامجي ألا يتوقف عن التكرار، أي لا أريد الخروج من تعليمة التكرار، بل أريد منه الانتقال بالتنفيذ إلى التكرار التالي مباشرةً دون تنفيذ بقية كتلة التعليمات.

ماذا نتوقع أن يكون خرج الكود التالي:

Python

```
for i in range(0,13 + 1):
    if ((i % 4 == 0) and (i > 4)):
        continue;
    print(" i = " , i);
```


ملاحظات:

لاحظ أن تنفيذ تعليمات الحلقة (إظهار قيمة i) جرى من أجل القيم من 1 حتى 7، ولكن عند القيمة 8 (مضاعفات 4) لم يتم التنفيذ/الإظهار بل تابعنا مباشرة اختبار شرط التكرار نتيجة تنفيذ **continue**، هنا صارت $i=9$ ، شرط الحلقة صحيح، إذاً ننفذ تعليمة الحلقة أي إظهار i ، وهكذا يتكرر الإظهار للقيم 9, 10, 11. ثم تأخذ i القيمة 12 (عدد من مضاعفات 4)، إذاً تنفذ **continue**، فننتقل مباشرة إلى اختبار شرط الحلقة مع القيمة 13.

نشاط: ناقش خرج الكود التالي:

```
10 # continue
11 for status in watches:
12     if status == "faulty":
13         print("Found faulty watch, skipping ...")
14         continue
15     print(f"This watch is {status}")
16     print("Shipping new watch to the customer")
```

الخرج:

```
This watch is ok
Shipping new watch to the customer
This watch is ok
Shipping new watch to the customer
This watch is ok
Shipping new watch to the customer
Found faulty watch, skipping ...
This watch is ok
Shipping new watch to the customer
This watch is ok
Shipping new watch to the customer
```

تمرين:

أكتب برنامج بلغة بايثون يطلب إدخال كلمة المرور الصحيحة و بعد أقصى 3 محاولات:

- يطلب البرنامج إدخال كلمة المرور
- إذا كانت كلمة المرور صحيحة يطبع الرسالة `Correct password` و إلا فإنه يطلب إدخال كلمة المرور مرة أخرى
- بعد المحاولة الثالثة يتوقف و يطبع الرسالة `No more attempts`

ناقش خرج الكود البرمجي التالي بلغة بايثون:

```
password checker.py > ...
1  real_password = "Admin123"
2
3  attempts = 3
4  while attempts > 0 :
5
6      input_password = input("Enter the password: ")
7      attempts -= 1
8
9      if real_password == input_password :
10         print("Correct password")
11         break
12     else:
13         print("No more attempts")
```

2. الأنماط الأساسية

يوجد في بايثون أنماط معرفة بسيطة (`int`, `float`, `complex`, `string`, `bool`)، وأنماط مركبة (`list`, `set`, `tuple`, `dict`: dictionary).

- لا يوجد ترميز صريح: أي أنه لا يمكن تعريف نمط متحول بشكل صريح قبل استخدام المتحول.!!

مثال:

```
# assign integer
i = 34
# assign float
f = 105.7
# assign complex
c = 5+4j
# assign Boolean
b = True
# assign string
s = "This is a string"
```

- لا يوجد نمط دائم للمتحول في بايثون كما في لغات البرمجة التي نعرفها.
- يتغير النمط بتغير القيمة التي تُسند إلى المتحول!
- يمكننا الاستفسار عن نمط المتحول في أي وقت، بالتابع **type**.

مثال:

```
# assign integer
a = 34
print( " a=",a, type(a) )
# assign float
a= 105.7
print( " a=",a, type(a) )
# assign Boolean
a = True
print( " a=",a, type(a) )
# assign string
a= "This is a string"
print( " a=",a, type(a) )
```

1.2. الأنماط العددية

- كما في لغات C-like نرى وجود نمط الأعداد الصحيحة **int** ونمط الأعداد العشرية **float**.
والعمليات على هذه الأنماط، هي التي ذكرناها في تعريفنا لتعليمة الإسناد حيث يجري الإسناد لتعبير حسابي (التعبير الحسابي هو تركيب من: ثوابت، متحولات، توابع عددية وعمليات حسابية).
- يوجد في بايثون نمط الأعداد العقدية **complex**.

التوابع الحسابية مسبقة التعريف

- من التوابع الجاهزة تابع القيمة المطلقة، تابع التقريب إلى عدد صحيح، تابع الرفع إلى قوة.
abs(), **round()**, **pow()**
- التوابع **sum()**, **min()**, **max()** يمكن أن تأخذ عدد متغير من الوسائط.
sum(10, 15, 3) # 28
sum(15) # 15

2.2. النمط المنطقي

ويأخذ أحد قيمتين **True**, **False**.

العمليات المنطقية:

وهي العمليات المعروفة **and**, **or**, **not**.

إظهار جدول العمليات المنطقية **Truth Table**

Python

```
# testing the conditional AND operators (&&)
print( "Conditional AND (and)" +
      "\nfalse and false: " + str(False and False) +
      "\nfalse and true: " + str(False and True) +
      "\ntrue and false: " + str(True and False) +
      "\ntrue and true : " + str(True and True))

# testing the conditional OR operators (||)
print( "\n\nConditional OR (or)" +
      "\nfalse or false: " + str(False or False) +
      "\nfalse or true: " + str(False or True) +
      "\ntrue or false: " + str(True or False) +
      "\ntrue or true : " + str(True or True))

# testing the logical AND operators (&)
print( "\n\nLogical AND (&)" +
      "\nfalse & false: " + str(False & False) +
      "\nfalse & true: " + str(False & True) +
      "\ntrue & false: " + str(True & False) +
      "\ntrue & true : " + str(True & True))

# testing the logical OR operators (|)
print( "\n\nLogical OR (|)" +
      "\nfalse | false: " + str(False | False) +
      "\nfalse | true: " + str(False | True) +
      "\ntrue | false: " + str(True | False) +
      "\ntrue | true : " + str(True | True))

# testing the logical exclusive OR operators (^)
print( "\n\nLogical Exclusive OR (^)" +
      "\nfalse | false: " + str(False ^ False) +
      "\nfalse | true: " + str(False ^ True) +
      "\ntrue | false: " + str(True ^ False) +
      "\ntrue | true : " + str(True ^ True))

# testing the logical NOT operators (not)
print( "\n\nlogical NOT (not)" +
      "\nnot false: " + str(not False) +
      "\nnot true : " + str(not True))
```

العمليات المنطقية على البتات Bitwise Operation

نعلم أن التمثيل للمعطيات في الحاسب يكون بالتمثيل الثنائي.

فإذا أخذنا التمثيل على 8 بت (8bit).

يكون التمثيل للأعداد التالية كما يلي:

65=01000001

35=00100011

تسمح لنا لغة بايثون بكتابة الأعداد بنظام العد الثنائي، الثماني، الست عشري `.hex()`، `.oct()`، `.bin()`.

```
a = 35
print(bin(a))
print(oct(a))
print(hex(a))
```

```
0b100011
0o43
0x23
```


العمليات على البتات:

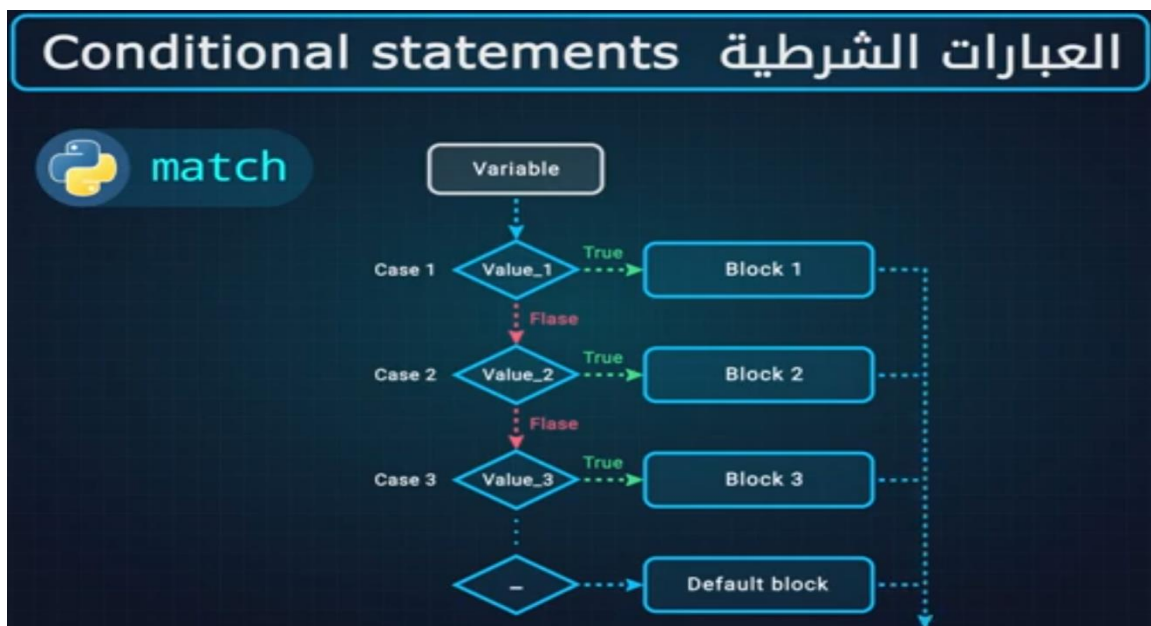
and: $0 \& 0 = 0$ $0 \& 1 = 0$ $1 \& 0 = 0$ $1 \& 1 = 1$
or: $0 | 0 = 0$ $0 | 1 = 1$ $1 | 0 = 1$ $1 | 1 = 1$
xor: $0 \wedge 0 = 0$ $0 \wedge 1 = 1$ $1 \wedge 0 = 1$ $1 \wedge 1 = 0$
 $65 \& 35 = 00000001 = 1$
 $65 | 35 = 01100011 = 99$
 $65 \wedge 35 = 01100010 = 98$

- **تنويه:** العمليات المنطقية bitwise operation (بت مقابل بت) متاحة في بايثون بنفس الطريقة الموجودة في لغات C-Like.

```
a = 65
b = 35
print(a, " & ", b, "=", a & b)
print(a, " | ", b, "=", a | b)
print(a, " ^ ", b, "=", a ^ b)
```

```
65 & 35 = 1
65 | 35 = 99
65 ^ 35 = 98
```

أمثلة إضافية على match-statement (موجودة في نسخ بايثون 3.10 وأحدث منها):



 Syntax

```
match var_name :
    case value_1 :
        Block_1
    case value_2 :
        Block_2
    ...
    case _ :
        Default_block
following_statement
```

نشاط:

أكتب برنامج بلغة بايثون يطلب من المستخدم إختيار اللغة عن طريق الضغط على الأرقام التالية:

- ① اللغة العربية
- ② اللغة الإنجليزية
- ③ اللغة الفرنسية

كود النشاط السابق بلغة بايثون مع عينة من الخرج:

```
match statement.py > ...
1  print("1 Arabic\n2 English\n3 Frensh")
2  choice = int(input("Chooce a number: "))
3  match choice :
4      case 1 :
5          print("مرحبا")
6      case 2 :
7          print("Hello")
8          print("How can we help you")
9      case 3 :
10         print("Bonjour")
11     case _ :
12         print("Unsupported language")
```

```
1 Arabic
2 English
3 Frensh
Choose a number: 1
مرحبا
```

مثال آخر: (الآلة الحاسبة البسيطة التي تم تنفيذها في نشاط سابق يمكن تنفيذها باستخدام match-statement)

```
calculator.py > ...
1  n1 = float(input("Enter the first number: "))
2  op = input("Enter an operator (+, -, *, /): ")
3  n2 = float(input("Enter the second number: "))
4  match op :
5      case "+":
6          print(n1 + n2)
7      case "-":
8          print(n1 - n2)
9      case "*":
10         print(n1 * n2)
11     case "/":
12         if n2 != 0 :
13             print(n1 / n2)
14         else:
15             print("Cannot divide by zero")
16     case _ :
17         print("Unkown operator")
```

عينة من خرج البرنامج:

```
Enter the first number: 3
Enter an operator (+, -, *, /): +
Enter the second number: 2.5
5.5
```

مثال إضافي:

أكتب برنامج بلغة بايثون يطلب من المستخدم إدخال وزنه (weight: kg) و طوله (height: cm)، ثم يقوم:

- إحتساب مؤشر كتلة الجسم (BMI: Body Mass Index) مع العلم أن:

$$\text{BMI} = \frac{\text{weight}}{\text{height}^2}$$

kg m

- تقييم الوزن مع العلم أن الشخص يعتبر:



```

weight evaluation.py > _
1 weight = float(input("Enter your weight(kg): "))
2 height = float(input("Enter your height(cm): "))
3 bmi = weight/(height/100)**2
4 if bmi < 18.5 :
5     print("Underweight")
6 elif bmi < 25 :
7     print("Healthy")
8 elif bmi < 30 :
9     print("Overweight")
10 else:
11     print("Obese")

```

```

Enter your weight(kg): 67
Enter your height(cm): 173
Healthy

```

3. فقط في بايثون

حساب قيمة أي تعبير حسابي يعطى كمدخل للبرنامج (استخدام `eval()`)
يقوم البرنامج التالي بإدخال أي تابعين حسابيين $f1(x)$, $f2(x)$ وإخراج جدول بقيمتهما.

في المثال $f2(x) = \text{pow}(x,3)$, $f1(x) = x**2$

- أليس رائعاً أن يمكننا برنامج بسيط مقارنة قيم أي تابعين حسابيين. أحسنت `eval()` صنعاً!

```

f1 = input("Function 1: f1(x) = ")
f2 = input("Function 2: f2(x) = ")
print("f1(x) ", f1)
print("f2(x) ", f2)
x0 = 0; dx = 0.25
for i in range(0, 10):
    x = x0 + i * dx
    y1 = eval(f1)
    y2 = eval(f2)
    print(" %4.2f %6.2f %6.2f"%(x, y1,y2))

```

```

Function 1: f1(x) = x**2
Function 2: f2(x) = pow(x,3)
f1(x)    x**2
f1(x)    pow(x,3)
0.00     0.00     0.00
0.25     0.06     0.02
0.50     0.25     0.12
0.75     0.56     0.42
1.00     1.00     1.00
1.25     1.56     1.95
1.50     2.25     3.38
1.75     3.06     5.36
2.00     4.00     8.00
2.25     5.06     11.39

```


التمرين 2: حل معادلة من الدرجة الثانية (a لا تساوي 0)

حل معادلات من الدرجة الثانية.

$$ax^2 + bx + c = 0$$

Python

```
import math
# Read a, b, c
Sa = input(" Input a = "); a = float(Sa)
Sb = input(" Input b = "); b = float(Sb)
Sc = input(" Input c = "); c = float(Sc)

delta = b*b - 4 * a * c
# compute the equation roots

if (delta < 0):
    print(" No Real Solution ")
if (delta == 0):
    print(" One Solution ")
    x = -b / (2 * a)
    print(" x= " + str(x));
if (delta > 0):
    print(" Two Solutions ")
    x1 = (-b - math.sqrt(delta)) / (2 * a)
    x2 = (-b + math.sqrt(delta)) / (2 * a)
    print(" x1= " , x1 , " x2 " , x2)
```

الكلمات المحجوزة المستعملة

and, break, continue, elif, else, False, for, if, import, in, not, or, while, True

الكلمات المحجوزة في بايثون Python reserve word

تحتوي لغة بايثون على 33 كلمة مفتاحية/ محجوزة فقط!

وكما في لغات C جميع الكلمات المحجوزة حروف صغيرة. ما عدا **False, True, None**

and	except	nonlocal
as	finally	not
assert	for	or
break	from	pass
class	global	raise
continue	if	return
def	import	try
del	in	while
elif	is	with
else	lambda	yield
False	None	True

توصيف عمل الكلمات المحجوزة في بايثون:

Description Python reserved word

Keyword	Description
<u>and</u>	A logical operator
<u>as</u>	To create an alias
<u>assert</u>	For debugging
<u>break</u>	To break out of a loop
<u>class</u>	To define a class
<u>continue</u>	To continue to the next iteration of a loop
<u>def</u>	To define a function
<u>del</u>	To delete an object
<u>elif</u>	Used in conditional statements, same as else if
<u>else</u>	Used in conditional statements
<u>except</u>	Used with exceptions, what to do when an exception occurs
<u>False</u>	Boolean value, result of comparison operations
<u>finally</u>	Used with exceptions, a block of code that will be executed no matter if there is an exception or not
<u>for</u>	To create a for loop
<u>from</u>	To import specific parts of a module
<u>global</u>	To declare a global variable
<u>if</u>	To make a conditional statement
<u>import</u>	To import a module
<u>in</u>	To check if a value is present in a list, tuple, etc.
<u>is</u>	To test if two variables are equal/identical
<u>lambda</u>	To create an anonymous function
<u>None</u>	Represents a null value
<u>nonlocal</u>	To declare a non-local variable
<u>not</u>	A logical operator
<u>or</u>	A logical operator
<u>pass</u>	A null statement, a statement that will do nothing
<u>raise</u>	To raise an exception
<u>return</u>	To exit a function and return a value
<u>True</u>	Boolean value, result of comparison operations
<u>try</u>	To make a try...except statement

<u>while</u>	To create a while loop
<u>with</u>	Used to simplify exception handling
<u>yield</u>	To end a function, returns a generator

نهاية المحاضرة 2